



Java et la Programmation Orientée Objet



Houssem MAHMOUDI
houssem.mahmoudi@ensicaen.fr

Ensicaen©2025 - HM



Chapitre 4 : Gestion de Fichiers et Sérialisation

1. Notion de flux
2. Classes de manipulation de flux
3. Création d'un fichier d'octets
4. Lecture d'un fichier d'octets
5. Création d'un fichier texte
6. Lecture d'un fichier texte
7. Mémoire tampon
8. Classe File
9. Sérialisation
10. Try avec ressources
11. StringBuilder

Ensicaen©2025 - HM

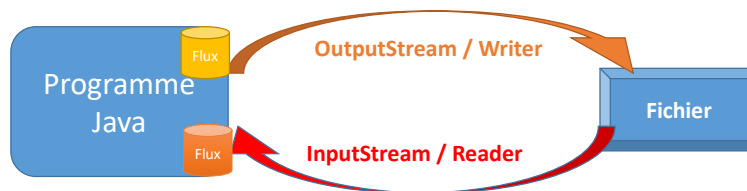
2

I. Notion de flux

- Un flux (ou Stream) est un **canal de communication** qui permet de gérer le transfert de données entre une source et une destination.
- D'une manière générale, on trouve deux sens de flux :
 - Flux d'entrée : permet à un programme de lire des données depuis une source.
 - Flux de sortie : permet à un programme d'écrire des données vers une destination.
- Java propose deux types de flux en fonction du type de données :
 - **Flux Binaires (d'Octets)** : L'information échangée est transmise sans modification entre la mémoire et la destination
 - **Flux Texte (Caractères)** : L'information échangée est transmise sous forme d'une suite de caractères.

II. Classes de manipulation de flux

- En Java, la gestion de flux se fait selon le type de données par les classes abstraites suivantes :
- **Flux d'octets (Binaire)** :
 - **InputStream** : permet de lire des données brutes (octets) depuis une source.
 - **OutputStream** : permet d'écrire des données brutes (octets) vers une destination.
- **Flux de caractères (Texte)** :
 - **Reader** : permet de lire des caractères (texte) depuis une source.
 - **Writer** : permet d'écrire des caractères (texte) vers une destination.



III. Création/Ecriture d'un fichier d'octets

- La classe **OutputStream** gère le flux de sortie est une classe abstraite.
- La classe `java.io.FileOutputStream` hérite de la classe `OutputStream`, permet d'écrire des données sous forme d'octets vers un fichier.
- Les constructeurs de la classe `FileOutputStream` sont :
 - **FileOutputStream**(String **name**) : Ouvre un flux de sortie vers un fichier spécifié par son nom ou chemin absolue. Si le fichier n'existe pas, il sera créé sinon, il sera écrasé.
 - **FileOutputStream**(String **name**, boolean **append**) : Ouvre un flux de sortie vers un fichier spécifié par son nom. Le paramètre *append* détermine si les données sont ajoutées à la fin du fichier (**true**) ou si le fichier est écrasé (**false**).
 - **FileOutputStream**(File **file**) : Crée un flux de sortie vers un fichier spécifié par un objet `File`.

Exemple : `FileOutputStream fos = new FileOutputStream("entiers.dat");`

III. Création/Ecriture d'un fichier d'octets

- La classe `FileOutputStream` peut lever plusieurs exceptions :
 - **FileNotFoundException** : Cette exception est levée si le fichier ne peut pas être créé ou ouvert. Elle hérite de **IOException**. (*Checked*)
 - **SecurityException** : Si l'accès en écriture au fichier est interdit. (*Unchecked*)
- Les méthodes de la classe `FileOutputStream` sont rudimentaires :
 - `fos.write(byte [] b)` : Ecrire un tableau d'octets dans le fichier.
 - `fos.write(int b)` : Ecrire un entier (octet) dans le fichier. Les valeurs valides vont de 0 à 255 (les bits supérieurs sont ignorés). Elle peut lever l'exception **IOException**
- La classe **DataOutputStream** possède des méthodes plus évoluées. Elle permet d'écrire des données de types primitives (`int`, `float`, `boolean`,...) dans un flux de sortie.

↳ L'objet `DataOutputStream` est créé par l'intermédiaire de l'objet `FileOutputStream`

- Exemple : `DataOutputStream dos = new DataOutputStream(fos);`

III. Création/Ecriture d'un fichier d'octets

- On peut aussi créer un objet `DataOutputStream` de cette façon :
 → `DataOutputStream sortie = new DataOutputStream(new FileOutputStream("entiers.dat"));`
- Les méthodes de la classe `DataOutputStream` sont :
 - `writeInt(int v)` : Écrit un entier `int` (4 octets).
 - `writeLong(long v)` : Écrit une valeur `long` (8 octets).
 - `writeDouble(double v)` : Écrit une valeur `double` (8 octets).
 - `writeChar(int v)` : Écrit un caractère Unicode (2 octets).
 - `writeUTF(String s)` : Écrit une chaîne en UTF-8 (taille variable).
 - `int size()` : Retourne le nombre d'octets écrits dans le fichier depuis l'instanciation.
- L'exception **`IOException`** peut être levée (erreurs générales d'entrée/sortie).

IV. Lecture à partir d'un fichier d'octets

- La classe **`InputStream`** gère le flux d'entrée est une classe abstraite.
- La classe `java.io.FileInputStream` hérite de la classe `InputStream`, permet de lire des données sous forme d'octets à partir d'un fichier.
- Les constructeurs de la classe **`FileInputStream`** sont :
 - **`FileInputStream(String name)`** : Ouvre un flux d'entrée à partir du nom du fichier.
 - **`FileInputStream(File fichier)`** : Crée un flux d'entrée à partir de l'objet `File` représentant le fichier à lire.

Exemple :

```
FileInputStream fis = new FileInputStream("entiers.dat");
```

- Le fichier *entiers.dat* doit se trouver dans le répertoire de travail actuel de l'application ou dans la racine du projet (IDE).
- On peut indiquer le chemin absolu du fichier : (**Windows** → `C:\\TPJava\\TP4\\entiers.dat`), (**Linux** → `/home/user/data/entiers.dat`).

IV. Lecture à partir d'un fichier d'octets

- La classe FileInputStream peut lever plusieurs exceptions :
 - **IOException** : Cette exception peut être levée lors de la lecture du fichier en cas d'interruption E/S.
 - **FileNotFoundException** : Cette exception est levée si le fichier spécifié est introuvable ou ne peut pas être ouvert.
- Les méthodes de la classe FileInputStream sont rudimentaires :
 - `fis.read(byte [] b)` : permet de lire des octets à partir du fichier et de les stocker dans un tableau.
 - `int b = fis.read()` : permet de lire un octet depuis le fichier. Retourne un entier (int) représentant l'octet lu (0 à 255) ou **-1** si la fin du fichier est atteinte.

IV. Lecture à partir d'un fichier d'octets

- Il existe une classe **DataInputStream** qui possède des méthodes avancées. Elle permet de lire des données primitives à partir d'un flux d'entrée.
- Exemple : `DataInputStream dis = new DataInputStream(fis);`
 - ↳ L'objet « dis » est créé par l'intermédiaire d'un objet FileInputStream « fis »
- La classe **DataInputStream** peut lever plusieurs exceptions :
 - **IOException** : exception levée lors d'une erreur E/S.
 - **EOFException** : Lorsque la fin du fichier (EOF) est atteinte avant que toutes les données requises soient lues.

IV. Lecture à partir d'un fichier d'octets

- Les méthodes de la classe **DataInputStream** sont :
 - `readInt()` : Lit un entier int (4 octets).
 - `readLong()` : Lit une valeur long (8 octets).
 - `readDouble()` : Lit une valeur double (8 octets).
 - `readChar()` : Lit un caractère Unicode (2 octets).
 - `readUTF()` : lit une chaîne de caractères encodée en UTF-8 à partir de flux de données
 - `int available ()` : Retourne le nombre d'octets disponibles pour la lecture
 - `mark()` : Marque la position actuelle dans le flux.
 - `reset()` : Rétablit le flux à la dernière position marquée.
 - `markSupported` : Retourne un booléen indiquant si le flux prend en charge les opérations de marquage et de réinitialisation.

IV. Manipulation d'un fichier Binaire

Exemple 1 : Ecrire un programme Java qui permet de cloner une image donnée n fois.

```
public class CopiesImage {
    public static void main(String[] args) {
        String oldpath = "C:\\tpjava\\moto.jpg";
        String newpath = "C:\\tpjava\\copies\\moto";
        Scanner sc = new Scanner(System.in);
        System.out.print("Nombre de copies : ");
        int n = sc.nextInt();
        FileInputStream fis;
        FileOutputStream fos;
        System.out.println("Copiage en cours ... ");
        try {
            for(int i = 1; i <= n; i++) {
                fis = new FileInputStream(oldpath);
                fos = new FileOutputStream(newpath + "_" + i + ".jpg");
                int x;
                while ((x = fis.read()) != -1) {
                    fos.write(x);
                }
                fis.close();
                fos.close();
            }
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Copiage terminé !! ");
    }
}
```

IV. Manipulation d'un fichier Binaire

Exemple 2 : Ecrire un programme Java qui permet de saisir des entiers non nuls depuis le clavier et de les enregistrer dans un fichier binaire.

Que remarquez-vous ?

```
import java.io.*;
import java.util.Scanner;
public class EntiersDansFichier {
    public static void main(String[] args){
        try {
            FileOutputStream fos = new FileOutputStream("C:\\TPJava\\entiers.dat");
            System.out.println("Ecriture dans le fichier");
            Scanner s = new Scanner(System.in);
            int x;
            do {
                System.out.print("donner un entier: "); x = s.nextInt();
                if(x!=0)
                    fos.write(x);
            }while(x!=0);
            fos.close();
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        try {
            FileInputStream fis = new FileInputStream("C:\\TPJava\\entiers.dat");
            System.out.println("Lecture a partir du fichier");
            int a;
            while ((a = fis.read()) != -1) {
                System.out.println("--> " + a);
            }
            fis.close();
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

IV. Manipulation d'un fichier Binaire

Exemple 3 : même énoncé, utiliser la classe DataInputStream et DataOutputStream

```
import java.io.*;
import java.util.Scanner;
public class CreationFichierData {
    public static void main(String[] args) throws Exception{
        FileOutputStream fos = new FileOutputStream("C:\\TPJava\\TP4\\entiers.dat");
        DataOutputStream dos = new DataOutputStream(fos);
        System.out.println("Ecriture dans le fichier");
        Scanner s = new Scanner(System.in);
        int x;
        do {
            System.out.print("donnez un entier: "); x = s.nextInt();
            if(x!=0)
                dos.writeInt(x);
        }while(x!=0);
        dos.close();
        fos.close();
        System.out.println("Lecture a partir du fichier");
        FileInputStream fis = new FileInputStream("C:\\TPJava\\TP4\\entiers.dat");
        DataInputStream dis = new DataInputStream(fis);
        System.out.println("Available: " + dis.available());
        int a;
        while (dis.available()>0) {
            a = dis.readInt();
            System.out.println("--> " + a);
        }
        dis.close();
        fis.close();
    }
}
```

V. Création d'un fichier texte

- La classe abstraite **Writer** du package **java.io** est la classe de base pour toutes les classes permettant d'écrire des caractères dans un fichier texte.
- La classe **FileWriter**, dérivée de **Writer**, permet d'écrire des caractères dans un fichier texte. Elle peut lever une exception **IOException**.
- Exemple : `FileWriter fw = new FileWriter("monFichier.txt");` crée un nouveau fichier s'il n'existe pas, sinon il écrase le fichier existant.
- Les constructeurs de la classe **FileWriter** sont :
 - `FileWriter(String nomFichier)` : Créer un flux texte en donnant son nom.
 - `FileWriter(String nomFichier, boolean append)` : Créer un flux texte en donnant son nom et la possibilité d'ajouter des données à la fin du fichier.
 - `FileWriter(File fichier)` : Créer un flux texte en se basant sur la classe **File** (voir plus loin).

V. Création d'un fichier texte

- Les méthodes de la classe **FileWriter** sont :
 - `write(int c)` : Ecrit un seul caractère.
 - `write(char[] t)` : Ecrit un tableau de caractères.
 - `write(String str)` : Ecrit une chaîne de caractères.
 - `flush()` : Vide le tampon et écrit immédiatement dans le fichier.
 - `close()` : Ferme le flux et libère les ressources
- On peut utiliser la classe **PrintWriter** pour écrire de chaînes formatés dans un fichier texte avec la méthode `printf()`
- Exemple :

```
PrintWriter pw = new PrintWriter(new FileWriter("calcul.txt"));
pw.printf("%s a pour côté %d \n", name, val);
pw.println(15);
pw.println("Bonjour");
```

VI. Lecture d'un fichier texte

- La classe abstraite **Reader** du package **java.io** est la classe de base pour toutes les classes permettant de lire des caractères à partir d'un fichier texte.
- La classe **FileReader**, dérivée de **Reader**, permet de lire des caractères à partir d'un fichier texte.
- Elle peut lever une exception **IOException** en cas d'erreur lors de l'accès au fichier.
- Exemple : `FileReader fr = new FileReader("monFichier.txt");`
- Si le fichier n'existe pas, l'exception **FileNotFoundException** sera levée.
- Les constructeurs de la classe **FileReader** sont :
 - `FileReader(String fileName)`
 - `FileReader(File file)`

VI. Lecture d'un fichier texte

- Les méthodes de la classe **FileReader** sont :
 - `int read(int c)` : Lit un seul caractère à la fois depuis le fichier. (Retourne **-1** si la fin de fichier est atteinte).
 - `int read(char[] t)` : Lit un nombre de caractères définis par le tableau.
 - `long skip(long n)` : Ignorer n caractères du fichier. (Sauter des parties du fichier).
 - `close()` : Ferme le flux de lecture.

Exercice 1

- On souhaite écrire un programme Java permettant d'écrire des lignes dans un fichier texte sous la forme « **n** a pour carré **carre_de_n** ».
- Les entiers **n** saisis sont non nuls.
- La saisie de 0 arrête l'opération.
- Sauvegarder **n** et son carré dans un fichier texte comme mentionné dans l'énoncé
- Afficher le contenu du fichier texte.
- Utiliser la classe **StringBuilder** ou **StringBuffer** (Thread Safe) pour la manipulation de chaînes de caractères.

VII. Mémoire tampon

- Un tampon (Buffer) est un espace mémoire utilisé pour optimiser les échanges de flux avec un fichier.
- Les données sont d'abord stockées dans la mémoire tampon, puis envoyées au fichier lorsqu'il est plein (**flush**). Dans le but :
 - Optimisation des ressources : L'accès au disque est plus lent que l'accès à la mémoire.
 - Réduction du nombre d'accès au fichier : Écrire en bloc plutôt qu'octet par octet permet de réduire le nombre d'opérations d'E/S.
- Pour optimiser l'écriture dans un fichier, on utilise **FileOutputStream** avec la classe **BufferedOutputStream**.
- Pour optimiser la lecture depuis un fichier, on utilise **FileInputStream** avec la classe **BufferedInputStream**.

VII. Mémoire tampon – Fichiers d'octets

- La classe **BufferedOutputStream** est utilisée par la méthode suivante :

```
BufferedOutputStream bos = new BufferedOutputStream(new FileOutputStream(nomfichier) );
```

- La classe **BufferedInputStream** est utilisée par la méthode suivante :

```
BufferedInputStream disb = new BufferedInputStream(new FileInputStream(nomfichier) );
```

- On peut utiliser les deux classes Data(In/Out)putStream avec Buffered(In/Out)putStream

↳ `DataOutputStream dos = new DataOutputStream(bos);`

↳ `DataInputStream dis = new DataInputStream(bis);`

- Par défaut, la taille de la mémoire tampon est 8 Ko, on peut la personnaliser :

↳ `BufferedOutputStream bos = new BufferedOutputStream(fos, 16 * 1024); //16 Ko`

VII. Mémoire tampon – Ecriture des données

```
public class TamponEcriture {
    public static void main(String[] args) {
        String name = "Langages.dat";
        String [] data = {"Java", "Python", "C++", "PHP", "JavaScript"};
        try {
            FileOutputStream fos = new FileOutputStream(name);
            BufferedOutputStream bos = new BufferedOutputStream(fos, 16 * 1024);
            for(String s : data) {
                bos.write(s.getBytes());
                bos.write('\n');
            }
            bos.flush();
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Ecriture dans le fichier "+name+ " est terminé !");
    }
}
```

VII. Mémoire tampon – Lecture des données

```
public class TamponLecture {
    public static void main(String[] args) {
        String name = "Langage.dat";
        try {
            FileInputStream fis = new FileInputStream(name);
            BufferedInputStream bis = new BufferedInputStream(fis);
            while(bis.available() > 0) {
                System.out.print((char) bis.read());
            }
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        System.out.println("Lecture du fichier "+name+" est terminé !");
    }
}
```

VII. Mémoire tampon – Fichiers texte

- La classe **BufferedReader** est une classe du package java.io qui permet de lire un fichier texte en utilisant un tampon (buffer).
- Elle lit les données par blocs plutôt que caractère par caractère (améliorer les performances)
- Offre la méthode **readLine()** pour lire le fichier ligne par ligne.

```
BufferedReader bur = new BufferedReader(new FileReader("data.txt"));
String ligne;
while((ligne = bur.readLine()) != null) {
    System.out.println(ligne);
}
bur.close();
```

VII. Mémoire tampon – Fichiers texte

- La classe **BufferedWriter** est une classe du package java.io qui permet d'écrire dans un fichier texte en utilisant un tampon (buffer).
- Les caractères sont stockés dans un buffer puis écrits dans le fichier par blocs (réduit les accès au disque et améliore les performances).
- Offre la méthode **write()** pour lire le fichier ligne par ligne.

```
BufferedWriter buw = new BufferedWriter(new FileWriter("data.txt"));
buw.write("Bonjour");
buw.close();
```

VIII. La Classe File

- La classe **File** (package java.io) permet de manipuler des fichiers et des répertoires.
- Elle ne permet pas de lire ou écrire des données dans un fichier, mais elle offre des méthodes pour :
 - Vérifier l'existence d'un fichier.
 - Récupérer des informations sur un fichier (taille, chemin, permissions...).
 - Créer, renommer ou supprimer un fichier/dossier.
- Les constructeurs de la classe **File** sont :
 - File(String pathname); ➤ Crée un objet File à partir d'un chemin donné (relatif ou absolu).
 - File(String parent, String child); ➤ Construit un objet File en combinant le chemin du répertoire parent avec le nom de fichier ou le dossier child.

VIII. La Classe File

- Méthodes de la classe File sont :
 - `boolean exists()` : Vérifie si le fichier ou le dossier existe.
 - `boolean isFile()` : Vérifie si l'objet est un fichier.
 - `boolean isDirectory()` : Vérifie si l'objet est un répertoire.
 - `boolean createNewFile()` : Crée un fichier vide (si inexistant).
 - `boolean mkdir()` : Crée un dossier.
 - `String getName()` : Retourne le nom du fichier/dossier
 - `String getAbsolutePath()` : Retourne le chemin absolu
 - `long length()` : Retourne la taille du fichier (en octets)
 - `String[] list()` : Liste le contenu d'un répertoire
 - `boolean setReadable(boolean)` : Modifie la permission de lecture d'un fichier/dossier
 - `long lastModified()` : Retourne la date et l'heure de la dernière modification d'un fichier ou d'un dossier

IX. La Sérialisation

Problématique :

Comment stocker un objet dans un fichier et le transmettre ?

- Solution préliminaire :

On peut stocker tous les champs d'un objet dans un fichier (sauvegarder un par un) .

Mais les objets en Java sont complexe (Composition / Tableau / Map (paires clé-valeur) !!

- La **sérialisation** est le processus qui permet de convertir un objet Java en un flux d'octets, afin de le stocker de façon permanente (fichier : texte, JSON, XML ou une base de données) ou de le transmettre sur un réseau.
- La **désérialisation** est le processus inverse, où un flux d'octets est converti en un objet Java.

IX. La Sérialisation

Pour sérialiser un objet :

1. Implémenter l'interface **Serializable** : Pour qu'un objet puisse être sérialisé, il faut que sa classe implémente l'interface `java.io.Serializable`
2. Utiliser la classe **ObjectOutputStream** pour sérialiser l'objet.
3. Utiliser la méthode `writeObject` :

```
public final void writeObject(Object o) throws IOException
```

Pour désérialiser un objet :

1. Utiliser la classe **ObjectInputStream** pour désérialiser l'objet.
2. Utiliser la méthode `readObject` :

```
public final Object readObject() throws IOException, ClassNotFoundException
```

Remarque :

- ❖ Pour ne pas sérialiser un champ particulier, il faut le marquer avec le mot-clé **transient**.

IX. La Sérialisation – Exemple sérialisation

```
class Employee implements Serializable {
    protected String nom;
    protected String prenom;
    protected double salaire;
    public Employee(String nom, String prenom, double salaire) {
        this.nom = nom; this.prenom = prenom; this.salaire = salaire;
    }
}

public class SerializeEmployee {
    public static void main(String[] args) {
        Employee e[] = new Employee[3];
        e[0] = new Employee("Dupont", "John", 4500);
        e[1] = new Employee("Moulin", "Siwar", 3500);
        e[2] = new Employee("Dijou", "Sonia", 5200);
        try {
            FileOutputStream fos = new FileOutputStream("employee.ser");
            ObjectOutputStream oos = new ObjectOutputStream(fos);
            for (int i = 0; i < 3; i++) {
                oos.writeObject(e[i]);
            }
            oos.close();
            fos.close();
        } catch (FileNotFoundException x) {
            System.out.println(x.getMessage());
        } catch (IOException x) {
            System.out.println(x.getMessage());
        }
        System.out.println("Serialisation Terminée !");
    }
}
```

IX. La Sérialisation – Exemple désérialisation

```
class Employee implements Serializable {
    protected String nom;
    protected String prenom;
    protected double salaire;
    public Employee(String nom, String prenom, double salaire) {
        this.nom = nom;
        this.prenom = prenom;
        this.salaire = salaire;
    }
}

public class DeserializeEmployee {
    public static void main(String[] args) {
        Employee emp[] = new Employee[3];
        try {
            FileInputStream fis = new FileInputStream("employee.ser");
            ObjectInputStream ois = new ObjectInputStream(fis);
            for(int i=0; i<emp.length; i++) {
                emp[i] = (Employee) ois.readObject();
            }
            ois.close();
            fis.close();
        } catch (FileNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (ClassNotFoundException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println(e.getMessage());
        }
        for(int i=0; i<emp.length; i++) {
            System.out.println(emp[i].nom + " # " + emp[i].prenom + " # " + emp[i].salaire);
        }
    }
}
```

X. Try avec ressources

- *try-with-resources* est une structure introduite en Java 7.
- Permet de gérer automatiquement la fermeture des ressources (comme les fichiers, les connexions réseaux ou les bases de données) après utilisation.

▪ **Syntaxe :**

```
try (ResourceType resourceName = new ResourceType()) {
    // Code qui utilise la ressource
} catch (ExceptionType e) {
    // Gestion des exceptions
}
```

▪ **Exemple :**

```
try(FileInputStream fis = new FileInputStream(nameFile);
    DataInputStream dis = new DataInputStream(fis)) {
    //Manipulation du fichier avec l'objet dis
} catch (FileNotFoundException e1) {
    e1.printStackTrace();
} catch (IOException e1) {
    e1.printStackTrace();
}
```

XI. StringBuilder

- La classe String est immuable
- La classe StringBuilder est mutable, elle fait partie du package **java.lang**

Exemple :

```
StringBuilder sb1 = new StringBuilder();
StringBuilder sb2 = new StringBuilder("Bonjour");
```

Principaux méthodes de la classe StringBuilder :

- **append(valeur)** : ajouter à la fin de la chaîne

```
StringBuilder sb = new StringBuilder("Hello");
sb.append(" World");
```

- **insert(pos, valeur)** : insérer une chaîne dans une position

```
sb.insert(5, " Java");
```

XI. StringBuilder

- **replace(deb, fin, valeur)** : remplacer une partie de la chaîne

```
sb.replace(0,5,"Hi");
```

- **delete(deb, fin)** : supprimer une partie de la chaîne

```
sb.delete(0,2);
```

- **reverse()** : inverser une chaîne

```
sb.reverse();
```

- **length()** : retourne la longueur de la chaîne

```
int taille = sb.length();
```

- **toString()** : convertit l'objet StringBuilder en une chaîne

```
String s = sb.toString();
```

Exercice 2

- Écrire un programme qui crée un fichier binaire « **personne.dat** » contenant, pour différentes personnes, les informations suivantes : nom, prénom et année de naissance.
- La saisie des informations s'effectue dans la fenêtre console, comme cet exemple :

```
Nom du fichier à créer : infos.dat
Personne 1 :
nom : Carre
Prenom : Thibault
annee naissance : 1997
.....
Personne 2 :
nom : Mitenne
Prenom : Thomas
annee naissance : 2001
Personne 6 :
**** fin création fichier ****
```

On proposera deux solutions :

1. Les informations relatives au nom et au prénom seront stockés dans le fichier sous la forme de chaînes de 20 caractères (complétées par des espaces à la fin).
2. Ces mêmes informations seront conservées sous la forme d'une chaîne codée dans le format UTF. Les espaces en fin de chaîne seront complétés par des points.

Exercice 2

- Solution 1 :

Dubois	Thibault	1988
Duchene	Alfred	1993
Mitenne	Thomas	2001
Dutronic	Jean Philippe	1980

- Solution 2 :

```
Dubois.....Thibault.....1988
Duchene.....Alfred.....1993
Mitenne.....Thomas.....2001
Dutronic.....Jean Philippe.....1980
```